

IES Abastos

Valencia

**MercaIntelligence: Sistema de
análisis, predicción y detección de
anomalías de precios sobre el
catálogo de Mercadona mediante
Big Data y Machine Learning**

Memoria del Proyecto de IA & Big Data

Daniel Porras Morales

Curso de Especialización de Inteligencia Artificial y Big Data

Memoria del Proyecto de IA & Big Data

IES Abastos. Curso 2025/26. Grupo 8IA. 28 de Mayo de 2026

Tutor/a: Mario Carbonell Izquierdo

Agradecimientos

En primer lugar, quiero expresar mi agradecimiento a mi tutor de proyecto, Mario Carbonell Izquierdo, por la valiosa orientación que me ha brindado a lo largo de todo el curso.

Asimismo, agradezco profundamente al equipo docente del Curso de Especialización en Inteligencia Artificial y Big Data del IES Abastos. Los conocimientos que me han transmitido durante este año han constituido la sólida base técnica sobre la que se ha construido este proyecto. En este punto, quiero hacer una mención especial a Vicent Pavel Tortosa Lorenzo y a María Consuelo Richart Zanon, por su constante apoyo, cercanía y preocupación hacia mí.

En el ámbito personal, quiero dar las gracias de todo corazón a mis padres, quienes me han dado todas las facilidades de este mundo para estudiar y perseguir mis metas. En especial, a mi mamá, por estar siempre para mí, por ser mi mayor apoyo en este mundo y por respaldarme de forma incondicional, guiándome con paciencia incluso cuando me equivoco para ayudarme a aprender de mis errores.

Un agradecimiento muy especial para Ana Belén Sedano Salón, ya que por ella tomé la decisión de venirme a vivir a Valencia, y por encargarse de entregar en secretaría aquellos papeles de la matrícula que a mí se me habían olvidado.

Por último, no quiero dejar de agradecer a toda mi familia y amigos por el apoyo, el cariño y la paciencia recibidos durante todo el desarrollo de este trabajo. A todos ellos, gracias.

ÍNDICE

1. IDENTIFICACIÓN Y OBJETIVOS DEL PROYECTO.....	3
1.1. Descripción General del proyecto.....	3
1.2. Objetivos del Proyecto.....	3
1.2.1. Objetivo General.....	3
1.2.2. Objetivos Específicos.....	4
1.3. Alcance y Limitaciones.....	5
1.3.1. Alcance.....	5
1.3.2. Limitaciones.....	5
1.4. Planificación Temporal.....	6
2. DISEÑO DEL PROYECTO.....	7
2.1. Tecnologías Utilizadas.....	7
2.2. Arquitectura del Sistema.....	9
2.2.1. Capa de Ingesta y Orquestación.....	9
2.2.2. Capa de Almacenamiento.....	9
2.2.3. Capa de Procesamiento y ETL.....	10
2.2.4. Capa de Machine Learning y Deep Learning.....	10
2.2.5. Capa de Visualización y Despliegue.....	11
2.3. Flujo de trabajo y procesamiento de datos.....	12
2.4. Modelo de Datos.....	12
3. DESARROLLO DEL PROYECTO.....	14
3.1. Pipeline ETL Incremental.....	14
3.2. Módulo de Detección de Anomalías.....	14
3.2.1. Z-Score Rolling (Baseline Estadístico).....	14
3.2.2. Isolation Forest.....	15
3.2.3. Autoencoder LSTM.....	15
3.2.4. Comparativa de Métodos.....	15
3.3. Módulo de Predicción.....	16
3.3.1. LSTM Clasificador de Cambios de Precio.....	16
3.3.2. XGBoost Regresor de Precio.....	16
3.3.3. Comparativa LSTM vs XGBoost vs Baseline.....	16
3.4. Módulo NLP y Análisis de Marcas.....	17
3.4.1. Generación de Embeddings Semánticos.....	17
3.4.2. Detección de Equivalencias Marca Propia ↔ Comercial.....	17
3.5. Módulo IPC Personalizado.....	17
3.6. Análisis Complementarios.....	18
3.6.1. Segmentación K-Means del Catálogo.....	18
3.6.2. Detección de Shrinkflation.....	18
3.6.3. Detector de Rotación del Catálogo.....	18
3.7. API Flask.....	19
3.8. Frontend Vue 3.....	19
3.9. Despliegue con Docker.....	20

4. EVALUACIÓN Y CONCLUSIONES FINALES.....	20
4.1. Resultados Obtenidos.....	20
4.1.1. Convergencia Intermodular.....	21
4.2. Análisis Crítico.....	22
4.2.1. Fortalezas.....	22
4.2.2. Limitaciones.....	22
4.3. Conclusiones.....	23
4.4. Líneas de Mejora y Trabajo Futuro.....	24
4.4.1. Corto Plazo.....	24
4.4.2. Medio Plazo.....	24
4.4.3. Largo Plazo.....	24
5. REFERENCIAS.....	24
Bibliografía académica y artículos.....	24
Documentación técnica oficial.....	25
Frameworks y herramientas.....	25
Modelos preentrenados.....	26
6. GLOSARIO DE TÉRMINOS Y ACRÓNIMOS.....	26

1. IDENTIFICACIÓN Y OBJETIVOS DEL PROYECTO

1.1. Descripción General del proyecto

MercalIntelligence es una plataforma integral de inteligencia de precios diseñada para monitorizar, analizar y predecir la evolución del catálogo online de Mercadona. El proyecto aborda un problema real tanto del consumidor como del analista de retail: la ausencia de herramientas automatizadas capaces de rastrear la evolución diaria de más de 5.000 productos a lo largo del tiempo, detectar anomalías en los precios, identificar equivalencias entre marcas propias y comerciales, y anticipar cambios futuros de forma cuantitativa.

El sistema se alimenta de un scraper externo que se ejecuta diariamente mediante GitHub Actions y que extrae el catálogo completo de Mercadona en formato CSV. Estos ficheros se procesan a través de un pipeline ETL incremental que genera un Data Lake en formato Parquet particionado por fecha, y se indexan en Elasticsearch para su exploración visual en Kibana.

Sobre esta base de datos, compuesta por 763.435 observaciones de 5.101 productos únicos recopiladas entre noviembre de 2025 y mayo de 2026, se ejecutan módulos de Machine Learning y Deep Learning que abarcan tres líneas de análisis:

- **Detección de anomalías:** mediante tres enfoques complementarios — Z-Score Rolling, Isolation Forest y Autoencoder LSTM — que capturan dimensiones distintas del comportamiento anómalo de los precios.
- **Predicción de precios:** un clasificador LSTM que anticipa cambios de precio a 7 días vista y un regresor XGBoost que estima el precio futuro, con integración intermodular donde la probabilidad del LSTM alimenta al XGBoost como feature.
- **Análisis semántico NLP:** un pipeline basado en **sentence-transformers** que empareja automáticamente productos de marca propia con sus equivalentes de marca comercial, cuantificando la brecha de precio.

El frontend, desarrollado en Vue 3, permite al usuario calcular un IPC (Índice de Precios al Consumo) personalizado ponderado por consumo real según cestas predefinidas, visualizar predicciones de coste futuro, consultar alertas de anomalías y shrinkflation, y explorar las equivalencias de marca. Todo el stack se despliega con Docker Compose en cuatro servicios independientes: Elasticsearch, Kibana, API Flask y Frontend Nginx.

El valor diferencial del proyecto reside en la convergencia intermodular: tres modelos independientes validan mutuamente sus hallazgos. La degradación temporal de XGBoost coincide exactamente con los picos de anomalías del Autoencoder, y la probabilidad de cambio generada por el LSTM aparece como feature relevante (top 11 en importancia SHAP) en el XGBoost, confirmando que los cambios detectados son genuinos y que los módulos no operan de forma aislada.

1.2. Objetivos del Proyecto

1.2.1. Objetivo General

El objetivo general del proyecto es diseñar, desarrollar y desplegar una plataforma de inteligencia de precios que permita la monitorización continua, el análisis automatizado y la predicción de la evolución del catálogo de Mercadona, integrando técnicas de Data Engineering, Machine Learning, Deep Learning y Procesamiento del Lenguaje Natural en un sistema cohesionado y accesible al usuario final.

1.2.2. Objetivos Específicos

Se definieron los siguientes objetivos específicos, cada uno asociado a un módulo o componente del sistema:

1. **Construir un pipeline ETL incremental y escalable** que procese diariamente los ficheros CSV del scraper, aplique limpieza y **feature engineering** en vuelo, y almacene los resultados en un Data Lake particionado en formato Parquet, eliminando los cuellos de botella de memoria del procesamiento monolítico.
2. **Implementar un módulo de detección de anomalías multi-enfoque** que combine un baseline estadístico (Z-Score Rolling con ventana de 14 días y umbral de 2,5 desviaciones típicas), un modelo de Machine Learning no supervisado (Isolation Forest con 200 estimadores) y un modelo de Deep Learning (Autoencoder LSTM con ventana de 14 días y umbral P99), demostrando su complementariedad mediante métricas de solapamiento (Jaccard).
3. **Desarrollar un sistema de predicción de cambios y precios futuros** que integre un clasificador LSTM binario (secuencias de 14 días → horizonte de 7 días) para anticipar cambios de precio, y un regresor XGBoost (21 features incluyendo la probabilidad del LSTM) para estimar el precio futuro, validado mediante walk-forward expansión con 3 folds temporales.
4. **Implementar un pipeline NLP de equivalencias semánticas** basado en el modelo pre-entrenado **paraphrase-multilingual-MiniLM-L12-v2** de **sentence-transformers**, que genere embeddings del catálogo, calcule la similitud coseno entre productos de marca propia y marca comercial, y cuantifique la brecha de precio por medida entre ambas.
5. **Diseñar y calcular un IPC personalizado** ponderado por consumo real del usuario, con perfiles predefinidos (familiar, estudiante, vegano, deportista, pareja) y predicción de coste futuro que combine la probabilidad de cambio del LSTM con la tendencia histórica.
6. **Desarrollar análisis complementarios** que enriquezcan la plataforma: segmentación K-Means del catálogo en perfiles de negocio, detección de shrinkflation (reduflación) mediante correlación precio-medida, y detección de rotación del catálogo (altas y bajas de productos).
7. **Construir una API REST con Flask** que exponga todos los módulos analíticos mediante endpoints documentados, y un frontend en Vue 3 con visualizaciones interactivas (ECharts) que permita al usuario final explorar el sistema sin conocimientos técnicos.

8. **Desplegar el stack completo con Docker Compose** en cuatro servicios independientes (Elasticsearch, Kibana, API Flask, Frontend Nginx), utilizando multi-stage build para optimizar el tamaño de las imágenes.

1.3. Alcance y Limitaciones

1.3.1. Alcance

El proyecto cubre el ciclo completo de un sistema de inteligencia de datos, desde la ingesta y almacenamiento hasta el análisis avanzado y la visualización:

- **Fuente de datos:** catálogo online de Mercadona, obtenido mediante un scraper externo ejecutado diariamente a través de GitHub Actions. El dataset abarca 176 snapshots diarios desde el 3 de noviembre de 2025 hasta el 20 de mayo de 2026 (aproximadamente 6,5 meses).
- **Volumen:** 763.435 observaciones correspondientes a 5.101 productos únicos, con un tamaño medio de catálogo de 4.340 productos por día.
- **Cobertura analítica:** detección de anomalías (3 métodos), predicción de precios (2 modelos), equivalencias NLP, IPC personalizado, segmentación, shrinkflation y rotación del catálogo.
- **Infraestructura:** despliegue local con Docker Compose.

1.3.2. Limitaciones

Se identificaron las siguientes limitaciones del sistema, documentadas con transparencia:

- **Fuente única de datos:** el sistema depende exclusivamente del catálogo de Mercadona. No se realiza comparativa con otros supermercados, lo que limita la generalización de los hallazgos.
- **Dependencia del scraper externo:** la calidad y continuidad de los datos depende de un scraper que opera con un código postal concreto. Las bajas de productos pueden reflejar cambios de stock regionales, no necesariamente decisiones a nivel nacional.
- **Equivalencia semántica $\neq$ equivalencia funcional:** el matching NLP por texto no distingue productos que comparten vocabulario pero tienen funciones distintas (por ejemplo, lavavajillas de uso diario frente a limpiamáquinas).
- **Persistencia imbatible en predicción:** dado que el 95,9% de los precios no cambian en 7 días, un predictor trivial que asigne el precio actual como precio futuro obtiene un R^2 de 0,9984. El XGBoost solo aporta mejora marginal (+4,2% MAPE) en el 4,1% de casos con cambio real.
- **Shrinkflation concentrada en producto fresco:** el 100% de los 20 casos de reduflación detectados se concentran en fruta, verdura y pescado, donde la variación de peso por lote es natural. No se detectaron casos en productos industriales envasados dentro de la ventana temporal analizada.
- **Limitación de hardware e infraestructura local:** El entrenamiento de los modelos de Deep Learning (Autoencoder LSTM y LSTM clasificador) se realizó de forma remota en Google Colab utilizando una GPU NVIDIA T4. Esta decisión responde a una doble limitación en el entorno local: por un lado, la incompatibilidad de

TensorFlow $\geq$ 2.11 con aceleración por GPU en Windows nativo; por otro, la restricción de hardware local, que cuenta con una GPU NVIDIA GTX 960 de 2 GB de VRAM, una capacidad de memoria de video insuficiente para albergar los mapas de activación y el tamaño de lote (batch size) requeridos durante el entrenamiento. En consecuencia, la inferencia local se configuró para ejecutarse exclusivamente en CPU.

1.4. Planificación Temporal

Sprint	Periodo	Objetivo	Entregables Principales
Sprint 1 ¹	Semana 1	Infraestructura de datos	Pipeline ETL incremental, Parquet particionado, Elasticsearch + Kibana operativos
Sprint 2	Semana 2	Detección de anomalías	Z-Score, Isolation Forest, Autoencoder LSTM, comparativa Jaccard, dashboard Kibana
Sprint 3	Semana 3	Predicción y NLP	LSTM clasificador, XGBoost regresor, embeddings semánticos, IPC personalizado, API Flask
Sprint 4	Semana 4	Visualización y frontend	4 dashboards Kibana, clustering K-Means, frontend Vue 3 completo
Sprint Plus 1	Semana 5	Análisis complementarios	Detector shrinkflation, detector rotación

¹ El Sprint 1 se extiende a lo largo de varios meses porque incluye la fase de recopilación diaria de datos mediante el scraper automatizado, que se ejecutó de forma continua desde el inicio del curso. Los sprints posteriores, de carácter más intensivo en desarrollo, se concentraron en un periodo más reducido.

Sprint	Periodo	Objetivo	Entregables Principales
			catálogo, indexación ES
Sprint Plus 2	Semana 6	Despliegue y documentación	Docker Compose 4 servicios, README, memoria TFE

2. DISEÑO DEL PROYECTO

2.1. Tecnologías Utilizadas

La selección tecnológica se realizó priorizando la interoperabilidad entre componentes, el rendimiento en el procesamiento de datos y la viabilidad de despliegue en un entorno local. Se organizan a continuación por categoría funcional:

Categoría	Tecnología	Versión	Justificación
Lenguaje principal	Python	3.11	Ecosistema dominante en Data Science y ML. Compatibilidad nativa con pandas, TensorFlow y scikit-learn.
Procesamiento de datos	pandas	2.x	Manipulación tabular con soporte nativo de Parquet.
	NumPy	—	Operaciones vectorizadas de alto rendimiento.
	PyArrow	—	Motor de lectura/escritura Parquet con predicate pushdown y particionado.
Machine Learning	scikit-learn	—	Isolation Forest, K-Means, métricas de evaluación y preprocesamiento.
	XGBoost	—	Regresor de precio futuro con 21 features.
Deep Learning	TensorFlow / Keras	≥ 2.11	Autoencoder LSTM y LSTM clasificador. Entrenamiento en Google Colab (GPU T4), inferencia local en CPU.

Categoría	Tecnología	Versión	Justificación
NLP	sentence-transformers	—	Modelo paraphrase-multilingual-MiniLM-L12-v2 para generación de embeddings multilingües.
Almacenamiento y búsqueda	Elasticsearch	7.17	Indexación de documentos, búsqueda full-text y agregaciones para Kibana. Single-node.
Visualización analítica	Kibana	7.17	Dashboards interactivos sobre los índices de Elasticsearch.
Backend API	Flask	≥ 3.1	Framework ligero para la API REST.
	Flask-CORS	≥ 6.0	Habilitación de peticiones cross-origin desde el frontend.
Frontend	Vue 3	—	Framework SPA con Composition API y lazy loading de vistas.
	Vite	—	Bundler con Hot Module Replacement para desarrollo rápido.
	ECharts	—	Librería de gráficos interactivos (series temporales, barras, dispersión).
	Axios	—	Cliente HTTP para la comunicación con la API.
Infraestructura	Docker + Docker Compose	—	Orquestación de 4 servicios (ES, Kibana, API, Frontend).
	Nginx	Alpine	Proxy inverso y servidor de estáticos en producción. Multi-stage build.
	GitHub Actions	—	Ejecución diaria automatizada del scraper externo.

Categoría	Tecnología	Versión	Justificación
Vigilancia de archivos	watchdog	—	Detección event-driven de nuevos CSV del scraper para ingesta en tiempo real.
Entrenamiento GPU	Google Colab	GPU T4	Entrenamiento de modelos DL. TensorFlow ≥ 2.11 no soporta GPU en Windows nativo.
Gestor de paquetes frontend	pnpm	—	Alternativa eficiente a npm con deduplicación de dependencias.

2.2. Arquitectura del Sistema

La arquitectura del sistema se organizó en cinco capas funcionales que separan responsabilidades y permiten la evolución independiente de cada componente:

2.2.1. Capa de Ingesta y Orquestación

La ingesta de datos se basa en un scraper externo que se ejecuta diariamente mediante un **workflow** de GitHub Actions. Este scraper extrae el catálogo completo de Mercadona (~4.340 productos/día) y genera un fichero CSV con la nomenclatura `YYYY-MM-DD\_Mercadona\_DanielPorrasMorales.csv`.

El componente `ingesta\_incremental.py` opera en dos modos:

- **Modo desarrollo** (`--csv`): procesamiento manual de uno o varios ficheros CSV especificados por línea de comandos.
- **Modo producción** (`--watch`): vigilancia continua de la carpeta del runner local mediante la librería **watchdog**, que detecta automáticamente la llegada de un nuevo CSV y dispara el pipeline de procesamiento sin intervención manual.

Cada fichero procesado se copia en `data/raw/` como respaldo inmutable del dato original.

2.2.2. Capa de Almacenamiento

El almacenamiento se estructura en un Data Lake con tres niveles:

- **Capa raw** (`data/raw/`): ficheros CSV originales del scraper. Se conservan sin modificar como trazabilidad del origen.
- **Capa procesada** (`data/processed/fecha=YYYY-MM-DD/`): datos limpios y enriquecidos en formato Apache Parquet, particionados por fecha. Este diseño permite escrituras $O(1)$ — cada día solo genera su propia partición sin necesidad de reescribir el histórico — y lecturas optimizadas mediante **predicate pushdown**.

- **Capa de estado** (`data/state/ultimo\_precio.parquet`): tabla auxiliar compacta que mantiene exclusivamente el último precio y fecha conocidos de cada producto. Permite calcular métricas incrementales (`variacion\_pct`, `dias\_sin\_cambio`) sin cargar todo el histórico en memoria.

Adicionalmente, Elasticsearch actúa como capa de búsqueda e indexación, albergando los datos procesados y los resultados de los modelos en índices dedicados (`mercadona-precios`, `mercadona-catalogo`, `mercadona-shrinkflation`, `equivalencias-nlp`).

2.2.3. Capa de Procesamiento y ETL

El pipeline ETL aplica cuatro fases secuenciales sobre cada CSV nuevo:

1. **Limpieza**: normalización de tipos (eliminación de símbolos `€`, casteo a `float64`/`int64`), conversión de textos a minúsculas, eliminación de duplicados por `(referencia, fecha)` y descarte de filas sin referencia o precio.
2. **Feature engineering**: detección de marca propia mediante regex sobre el título de producto, cálculo de `variacion\_pct` y `dias\_sin\_cambio` usando la tabla de estado, e imputación de `unidad\_medida` mediante **forward/backward fill**.
3. **Almacenamiento particionado**: escritura en Parquet con `partition\_cols=["fecha"]` y compresión Snappy. Idempotente: reprocesar el mismo día sobrescribe esa partición sin afectar al resto.
4. **Indexación en Elasticsearch**: carga masiva mediante la API `helpers.bulk` con chunks de 5.000 documentos e ID determinístico (`{referencia}\_{fecha}`) que garantiza la idempotencia.

2.2.4. Capa de Machine Learning y Deep Learning

Sobre los datos procesados se ejecutan tres familias de modelos:

Detección de anomalías (tres enfoques complementarios):

- Z-Score Rolling: baseline estadístico que detecta desviaciones temporales puntuales (ventana 14 días, umbral $2,5\sigma$).
- Isolation Forest: modelo no supervisado con 200 estimadores que identifica productos estructuralmente atípicos en el espacio multidimensional de 5 features.
- Autoencoder LSTM: red recurrente que aprende a reconstruir secuencias normales de 14 días y detecta anomalías por error de reconstrucción elevado (umbral P99).

Predicción de precios (dos modelos con integración inter-modular):

- LSTM clasificador binario: predice la probabilidad de cambio de precio a 7 días vista. Entrenado con **class_weight** para compensar el desbalance extremo (95,9% clase negativa).

- XGBoost regresor: estima el precio futuro utilizando 21 features, incluyendo la probabilidad de cambio del LSTM como variable de entrada.

Análisis semántico NLP:

- Pipeline basado en **sentence-transformers** con el modelo **paraphrase-multilingual-MiniLM-L12-v2** que genera embeddings de 384 dimensiones para cada producto del catálogo y calcula la similitud coseno entre productos de marca propia y marca comercial dentro de la misma subcategoría.

Los modelos de Deep Learning se entrenaron en Google Colab con GPU T4 debido a la incompatibilidad de TensorFlow ≥ 2.11 con GPU en Windows nativo. Los modelos exportados (`.keras`, `.pkl`) se cargan localmente para inferencia.

2.2.5. Capa de Visualización y Despliegue

La capa de visualización se compone de dos interfaces complementarias:

Frontend Vue 3: aplicación SPA (**Single Page Application**) con 8 vistas — Inició, Buscador de Productos, IPC Personalizado, Alertas de Anomalías, Análisis de Marcas, Catálogo (altas/bajas), Shrinkflation y Acerca del Proyecto. Las visualizaciones interactivas se implementan con ECharts. La comunicación con el backend se abstrae mediante Axios en un servicio centralizado (`.services/api.js`).

Kibana: dashboards analíticos sobre los índices de Elasticsearch para exploración libre de datos por parte de usuarios técnicos.

Despliegue Docker Compose: el stack se orquesta en 4 servicios independientes:

Servicio	Imagen	Puerto	Función
es01	elasticsearch:7.17.29	9200	Motor de búsqueda. Single-node con healthcheck.
kibana	kibana:7.17.29	5601	Dashboards analíticos. Depende de ES.
api	Build Dockerfile.api (Python 3.11-slim)	5000	API Flask. Monta data/ y models/ como volúmenes.
frontend	Build Dockerfile.frontend (multi-stage)	80	Vue 3 servido por Nginx como proxy inverso.

El frontend utiliza un **multi-stage build**: la primera etapa compila la aplicación Vue con Node 20 y pnpm, y la segunda etapa copia solo los estáticos generados (`.dist/`) a una imagen Nginx Alpine, reduciendo el tamaño de ~1 GB a ~30 MB. Nginx actúa como proxy inverso,

redirigiendo las peticiones `/api/*` al servicio Flask y sirviendo las rutas SPA mediante `try_files`.

2.3. Flujo de trabajo y procesamiento de datos

El flujo de datos del sistema sigue un recorrido lineal desde la extracción hasta la visualización, con bifurcaciones paralelas en la capa analítica:

1. **Extracción:** el scraper de GitHub Actions genera un CSV diario con ~4.340 productos (19 columnas originales: referencia, título, categoría, subcategoría, formato, precios, URLs de imagen, timestamp, etc.).
2. **Ingesta y transformación:** el pipeline ETL limpia, enriquece y particiona los datos. Se generan features derivadas (`variacion_pct`, `dias_sin_cambio`, `es_marca_propia`, `marca_propia`). El resultado se persiste en Parquet particionado y se indexa en Elasticsearch.
3. **Análisis y modelado:** sobre el Data Lake particionado se ejecutan en paralelo los módulos de ML/DL:
 - a. Los 3 detectores de anomalías generan ficheros `.parquet` con scores y etiquetas booleanas en `data/anomalias/`.
 - b. El LSTM clasificador genera predicciones históricas en `data/predicciones/lstm/`.
 - c. El XGBoost genera el modelo entrenado en `models/xgboost_precio.pkl`.
 - d. El pipeline NLP genera embeddings y equivalencias en `data/nlp/`.
 - e. Los análisis complementarios (K-Means, shrinkflation, catálogo) generan artefactos en sus directorios respectivos.
 - f. Scripts de post-indexación enriquecen los documentos de Elasticsearch con los resultados de ML (anomalías, equivalencias, IPC).
4. **Servicio y visualización:** la API Flask carga en memoria al arrancar el histórico completo, las predicciones LSTM, las equivalencias NLP y precalcula las tendencias de precios. Expone 11 endpoints REST consumidos por el frontend Vue 3 y opcionalmente por Kibana.

2.4. Modelo de Datos

El modelo de datos central del sistema es el Data Lake particionado en formato Apache Parquet. Cada partición diaria contiene las siguientes columnas:

Campo	Tipo	Descripción
referencia	int64	Identificador numérico único del producto en Mercadona.

Campo	Tipo	Descripción
titulo	string	Nombre completo del producto tal como aparece en el catálogo.
categoria	string	Categoría principal (ej. "fruta y verdura", "bodega").
subcategoria	string	Subcategoría dentro de la categoría principal.
formato	string	Descripción del formato/envase (ej. "Botella 1 L", "Pieza 250 g aprox.").
precio_actual	float64	Precio de venta vigente en euros.
precio_anterior	float64	Precio previo al último cambio (nulo en el 95,3% de las observaciones).
precio_por_medida	float64	Precio normalizado por unidad de medida (€/kg, €/L, €/ud).
unidad_medida	string	Unidad de referencia del precio por medida.
es_marca_propia	bool	Indicador de pertenencia a marcas propias de Mercadona.
marca_propia	string	Marca detectada: "hacendado", "deliplus", "bosque verde", "compy" o "comercial".
tieneprecio_anterior	bool	Indicador de evento de cambio de precio.
variacion_pct	float64	Variación porcentual respecto al último precio conocido (feature derivada).
dias_sin_cambio	int	Días transcurridos desde el último cambio de precio (feature derivada).
fecha	date	Fecha de extracción (clave de particionado).
timestamp	datetime	Marca temporal exacta de la extracción del scraper.

Además del modelo central, los módulos analíticos generan artefactos complementarios con esquemas específicos:

- **Anomalías** (`data/anomalias/`): cada detector añade su score numérico y etiqueta booleana (ej. `zscore`, `anomalia\_zscore`, `score\_if`, `error\_mse`).
- **Predicciones** (`data/predicciones/lstm/`): incluye `prob\_cambio\_lstm` (probabilidad del LSTM) y `prediccion\_binaria`.
- **Equivalencias NLP** (`data/nlp/equivalencias.parquet`): pares `(ref\_mp, ref\_com)` con similitud coseno, precios y brecha porcentual.
- **Modelos serializados** (`models/`): ficheros `.pkl` (XGBoost, Isolation Forest, encoders, umbral AE) y `.keras` (Autoencoder LSTM, LSTM clasificador).

En Elasticsearch, los datos se indexan con un mapping estricto que define tipos `keyword` para campos de agregación, `text` para búsqueda full-text en títulos, y tipos numéricos para métricas, evitando la explosión de campos del mapping dinámico.

3. DESARROLLO DEL PROYECTO

3.1. Pipeline ETL Incremental

El pipeline ETL constituye el eje vertebrador del sistema. Se diseñó en dos fases: una primera fase de consolidación histórica (`consolidar\_historico.py`) que procesó los 176 ficheros CSV acumulados, y una segunda fase incremental (`ingesta\_incremental.py`) que opera en tiempo real.

La fase incremental aplica cuatro operaciones secuenciales sobre cada CSV nuevo: limpieza de tipos (eliminación de símbolos `€`, casteo numérico, normalización de textos), **feature engineering** (detección de marca propia por regex, cálculo de `variacion\_pct` y `dias\_sin\_cambio` mediante la tabla de estado `ultimo\_precio.parquet`), escritura en Parquet particionado por fecha con compresión Snappy, e indexación masiva en Elasticsearch mediante la API `helpers.bulk` con ID determinístico `{referencia}\_{fecha}`.

La migración del modelo monolítico (`maestro.parquet`) al particionado por fecha se realizó cuando el volumen superó las 660.000 filas, eliminando el cuello de botella $O(N)$ al concatenar históricos. La tabla de estado auxiliar permite calcular las features incrementales en $O(1)$ respecto al histórico total.

El modo producción utiliza la librería **watchdog** para detectar automáticamente la llegada de nuevos CSV del scraper, implementando un patrón **event-driven** que no requiere programación de tareas cron.

3.2. Módulo de Detección de Anomalías

Se implementaron tres detectores de anomalías complementarios que atacan el problema desde dimensiones matemáticas distintas: temporal (Z-Score), espacial-multivariante (Isolation Forest) y secuencial (Autoencoder LSTM).

3.2.1. Z-Score Rolling (Baseline Estadístico)

El Z-Score Rolling actúa como detector de eventos temporales puntuales. Para cada producto, se calcula la media y desviación típica en una ventana deslizando de 14 días, y se

marca como anomalía toda observación cuyo precio se desvíe más de 2,5 desviaciones típicas respecto a su propia media local.

Se detectaron **3.813 anomalías** (0,50%) distribuidas en **1.683 productos**. Las categorías más afectadas fueron fruta y verdura (705 alertas) y agua y refrescos (441), lo que refleja la volatilidad natural de los precios frescos y las promociones estacionales en bebidas.

3.2.2. Isolation Forest

Isolation Forest opera en el espacio multivariante de 5 features (`precio\_actual`, `precio\_por\_medida`, `variacion\_pct`, `dias\_sin\_cambio`, `ratio\_vs\_media\_subcat`) con 200 estimadores y contaminación del 0,5%.

Se detectaron **3.818 anomalías** (0,50%) hiper-concentradas en solo **59 productos** (~65 anomalías/producto). La investigación manual de estos 59 productos confirmó que son productos estructuralmente premium en subcategorías baratas (jamón ibérico de bellota con ratio 11× sobre la media de su subcategoría, cochinillo asado con 8,9×, leche infantil especializada con 4×). El modelo no sobreajusta: identifica outliers legítimos.

3.2.3. Autoencoder LSTM

El Autoencoder LSTM se entrenó en Google Colab con GPU T4 sobre 642.396 secuencias normales (donde el precio permaneció estable). La arquitectura consta de dos capas LSTM en el encoder (64 y 32 unidades), un cuello de botella (`RepeatVector`) y dos capas LSTM en el decoder (32 y 64 unidades), sumando 63.171 parámetros. El entrenamiento convergió con **early stopping** en la época 18 con un MSE de validación de $2,78 \times 10^{-8}$.

En inferencia local (CPU), el modelo evaluó 692.816 secuencias. La distribución del error de reconstrucción resultó bimodal: error ≈ 0 para secuencias estables y error elevado para secuencias con cambios de precio. Se seleccionó el percentil 99 como umbral, produciendo **6.929 anomalías** (1,00%) en **1.300 productos**.

3.2.4. Comparativa de Métodos

El hallazgo más relevante es la complementariedad extrema entre los tres métodos, medida por el índice de Jaccard:

Par comparado	Jaccard	Interpretación
Z-Score ↔ IF	0,006	Solapamiento casi nulo
Z-Score ↔ AE	0,000	Sin solapamiento
IF ↔ AE	0,005	Solapamiento casi nulo

Cada método captura una dimensión distinta: Z-Score detecta fluctuaciones temporales rápidas, Isolation Forest identifica productos premium estables y posibles errores de catalogación, y el Autoencoder modela rupturas en el patrón secuencial. La complementariedad (Jaccard ≈ 0) confirma que no existe redundancia y justifica el uso de

los tres enfoques. **Los detalles del pipeline multi-modelo y los resultados globales se amplían en el Anexo A.**

3.3. Módulo de Predicción

3.3.1. LSTM Clasificador de Cambios de Precio

El LSTM clasificador aborda un problema de clasificación binaria con desbalanceo extremo (95,7% clase negativa). Recibe secuencias de 14 días con 5 features y predice si habrá un cambio de precio en los 7 días siguientes.

La arquitectura consta de dos capas LSTM (64 y 32 unidades), Dropout (0,3 y 0,2), Batch Normalization y salida sigmoide, con 30.497 parámetros. Se aplicó `class\_weight` con penalización 24,3× en la clase minoritaria. El split temporal estricto de 3 vías (70/15/15) evita la fuga de información.

El umbral de decisión se calibró analíticamente sobre la curva Precision-Recall del conjunto de validación, obteniéndose un threshold óptimo de 0,901 que maximiza el F1-Score. En test, el modelo alcanzó un AUC-ROC de **0,893** frente al 0,771 del baseline (Regresión Logística), detectando el 62,4% de los cambios reales.

Se exportaron **658.373 predicciones históricas** cubriendo 4.858 productos, con 31.054 alertas de cambio de precio.

3.3.2. XGBoost Regresor de Precio

El XGBoost regresor estima el precio futuro a 7 días utilizando 21 features organizadas en cinco familias: lags de precio, estadísticas rolling, contexto temporal, características del producto y la probabilidad de cambio del LSTM.

El modelo se entrenó con 437.053 muestras y se evaluó sobre 110.667 muestras de test temporal. Los resultados globales muestran un MAPE del 1,29% ($R^2 = 0,7993$).

La comparación con el baseline naive (`precio\_futuro = precio\_actual`) reveló que la persistencia obtiene métricas globales superiores (MAPE 0,46%, R^2 0,9984), dado que el 95,9% de los precios no cambian en 7 días. Sin embargo, en el subconjunto con cambio real (4,1% de las muestras), XGBoost mejora el MAPE en un **+4,2%** respecto al naive, confirmando que captura señales reales de cambio.

El análisis SHAP reveló que `precio\_lag\_1` concentra el 53% del impacto total y que `prob\_cambio\_lstm` aparece en la posición 9 de importancia, constituyendo una validación cruzada no planificada entre dos modelos independientes. **Los detalles de las métricas de entrenamiento, curvas de pérdida y validación Walk-forward se amplían en el Anexo C.**

3.3.3. Comparativa LSTM vs XGBoost vs Baseline

La walk-forward validation con 3 folds expansivos evidenció una degradación temporal del XGBoost (R^2 de 0,9984 $\rightarrow$ 0,8496 $\rightarrow$ 0,8042) que coincide exactamente con el pico de anomalías detectado por el Autoencoder en abril-mayo de 2026. Esta convergencia entre módulos independientes — el XGBoost se degrada justo cuando el Autoencoder detecta más anomalías — constituye el hallazgo intermodular más sólido del proyecto, confirmando que los cambios de precio detectados son genuinos.

Modelo	Tarea	Métrica principal	Valor
LSTM clasificador	¿Habrá cambio?	AUC-ROC	0,893
XGBoost regresor	¿Cuánto costará?	MAPE (cambios reales)	10,75%
Baseline naive	Persistencia	MAPE global	0,46%

3.4. Módulo NLP y Análisis de Marcas

3.4.1. Generación de Embeddings Semánticos

Se utilizó el modelo preentrenado **paraphrase-multilingual-MiniLM-L12-v2** de **sentence-transformers** para generar embeddings de 384 dimensiones para cada producto del catálogo. Dos decisiones de preprocesamiento fueron clave: la eliminación del nombre de marca del título antes de generar embeddings (para evitar que el coseno mida afinidad de marca en vez de similitud de producto) y la concatenación del formato al título (para distinguir entre el mismo producto en diferentes envases).

3.4.2. Detección de Equivalencias Marca Propia ↔ Comercial

El pipeline calcula la similitud coseno entre productos de marca propia y marca comercial dentro de la misma subcategoría, con un umbral mínimo de 0,75 y TOP_K=3 candidatos. Se exige un mínimo de 3 productos comerciales por subcategoría (`MIN_COMERCIALES=3`) para evitar el patrón de "monopolio de match".

Se obtuvieron **924 equivalencias** con una similitud media de **0,844**. De los 924 pares, 892 resultaron directamente comparables por precio/medida (misma unidad). La conclusión económica principal: la marca comercial es, en mediana, un **49,0% más cara por unidad de medida** que su equivalente de marca propia.

El cálculo de la brecha pasó por 4 iteraciones de refinamiento para resolver sesgos sucesivos (precio absoluto vs por medida, unidades incompatibles, monopolio de match), documentados con transparencia. **Los detalles metodológicos y los casos específicos investigados se amplían en el Anexo B.**

3.5. Módulo IPC Personalizado

Se implementó un Índice de Precios al Consumo personalizado que, a diferencia del IPC oficial (ponderado por consumo agregado de la población), permite al usuario definir su propia cesta con productos y cantidades mensuales reales. La metodología implementada es:

1. Se toma como fecha base la primera fecha del dataset (3 de noviembre de 2025).
2. Los pesos se calculan automáticamente: $\text{peso}_i = (\text{precio_base}_i \times \text{cantidad_mensual}_i) / \text{gasto_total}$.

3. Para cada producto se calcula el índice temporal: $\text{índice}_t = \text{precio}_t / \text{precio_base} \times 100$.
4. El IPC de la cesta se obtiene como media ponderada: $\text{IPC}(t) = \sum [\text{peso}_i \times \text{índice}_i(t)]$.

Se incluyen 6 perfiles predefinidos (familiar, estudiante, vegano, deportista, pareja y una cesta personal del autor con productos reales de tickets de abril de 2026). La predicción de coste futuro combina la probabilidad de cambio del LSTM con la tendencia histórica (mediana de cambios pasados) de cada producto: $\text{precio_predicho} = \text{precio_actual} \times (1 + \text{prob_cambio} \times \text{mediana_cambio} / 100)$.

3.6. Análisis Complementarios

3.6.1. Segmentación K-Means del Catálogo

Se aplicó K-Means sobre 7 features estadísticas derivadas del comportamiento de precios (precio medio, desviación, rango, número de cambios, proporción estable, variación media y proporción de marca propia). Se eliminaron los 52 productos del percentil 99 (outliers extremos) y se aplicó transformación logarítmica a las variables asimétricas antes del escalado estándar.

Se seleccionó K=5 por criterio de negocio (el método del codo y el punto de inflexión de inercia), obteniendo un Silhouette Score de 0,441 y una varianza explicada PCA de 76,0%. Los 5 perfiles descubiertos — frescos de alta volatilidad (30 productos), básicos estables de marca blanca (2.942), marcas nacionales en promoción (120), dinámicos de variabilidad moderada (490) y estables de precio medio (1.467) — representan arquetipos de negocio accionables. **Los detalles, métricas de validación y gráficas de dispersión se amplían en el Anexo D.**

3.6.2. Detección de Shrinkflation

El detector de reduflación analiza la correlación inversa entre el precio absoluto y el precio por unidad de medida. Se clasifica como shrinkflation todo producto cuya variación de precio absoluto sea inferior al 5% pero cuya variación del precio por medida supere el 8% en una ventana de 15 días. La severidad se define como $S = \text{variación_medida} - \text{variación_precio}$. Se detectaron **20 casos únicos**, todos en producto fresco de marca comercial (85% fruta y verdura, 15% marisco y pescado), con una severidad media de 12,49 y variación media del precio por medida de +12,39%. El caso más severo fue la alcachofa (severidad +31,6: pieza de 200 g reducida a 150 g sin cambio de precio).

3.6.3. Detector de Rotación del Catálogo

El módulo detecta altas y bajas de productos mediante el análisis de presencias diarias, con dos filtros metodológicos para evitar falsos positivos:

- **Periodo de quemado (burn-in) de 10 días:** excluye productos que aparecen por primera vez en los primeros días del scraper (el catálogo base aún se estaba mapeando).
- **Ventana de confirmación de baja de 15 días:** solo se declara como descatalogado un producto ausente durante 15 días consecutivos.

Se detectaron **615 productos nuevos** y **717 descatalogados confirmados**. Las marcas propias representan el 63,1% de los nuevos lanzamientos, siendo cuidado facial y corporal (Deliplus) la categoría con mayor innovación (96 altas). El pico de lanzamientos se concentró en marzo de 2026 (+157 productos), coincidiendo con la campaña de primavera.

3.7. API Flask

La API REST se implementó con Flask y expone 11 endpoints que sirven como capa de acceso a todos los módulos analíticos. Al arrancar, carga en memoria el histórico completo, las predicciones LSTM, las equivalencias NLP y precalcula las tendencias de precios, priorizando la latencia de respuesta sobre el consumo de RAM (enfoque adecuado para el volumen del proyecto: ~4.300 productos). Los endpoints principales son:

Endpoint	Método	Función
/api/productos	GET	Búsqueda filtrada del catálogo (categoría, marca, texto)
/api/ipc	POST	Cálculo del IPC personalizado de una cesta
/api/ipc/prediccion	POST	Predicción de coste futuro (LSTM + tendencia)
/api/recomendaciones	POST	Alternativas más baratas vía equivalencias NLP
/api/equivalencias	GET	Pares marca propia ↔ comercial con similitud y brecha
/api/anomalias/hoy	GET	Productos anómalos en la última fecha (3 métodos)
/api/catalogo/eventos	GET	Productos nuevos y descatalogados
/api/shrinkflation	GET	Alertas de reduflación con severidad
/api/producto/<ref>	GET	Detalle completo de un producto (historial, anomalías, equivalencias)
/health	GET	Healthcheck con métricas del sistema

3.8. Frontend Vue 3

El frontend se desarrolló como una aplicación SPA (**Single Page Application**) con Vue 3 y Composition API, utilizando Vite como bundler y ECharts para las visualizaciones interactivas. La comunicación con la API se centraliza en un servicio Axios (`services/api.js`) y el enrutamiento se gestiona con **vue-router** con **lazy loading** de vistas. Las 8 vistas implementadas son:

- **Inicio (Home):** dashboard resumen con métricas globales del sistema.
- **Buscador de Productos:** búsqueda por texto, categoría y marca con detalle de cada producto.
- **IPC Personalizado:** constructor de cestas con perfiles predefinidos, gráfico de evolución temporal y predicción de coste futuro.
- **Alertas de Anomalías:** productos anómalos del día con filtro por método de detección.
- **Análisis de Marcas:** exploración de equivalencias NLP con brecha de precio y recomendaciones de ahorro.
- **Catálogo:** visualización de altas y bajas de productos.
- **Shrinkflation:** tabla de alertas de reduflación con severidad y detalle de transición de formato.
- **Acerca del Proyecto:** documentación técnica del sistema.

3.9. Despliegue con Docker

El despliegue se orquesta mediante Docker Compose con 4 servicios definidos en un único fichero ``docker-compose.yml``. Las dependencias entre servicios se gestionan mediante **healthchecks**: Kibana y la API esperan a que Elasticsearch esté operativo, y el frontend espera a que la API pase su **healthcheck**.

El frontend utiliza un **multi-stage build** en dos etapas: la primera (``node:20-alpine``) compila la aplicación Vue con pnpm, y la segunda (``nginx:alpine``) copia solo los estáticos generados (``dist/``), reduciendo la imagen de ~1 GB a ~30 MB. Nginx actúa simultáneamente como servidor de estáticos y como proxy inverso, redirigiendo las peticiones ``/api/*`` al servicio Flask internamente dentro de la red Docker (``mercai-net``).

La API monta ``data/`` y ``models/`` como volúmenes externos, permitiendo actualizar los datos y modelos entrenados sin necesidad de reconstruir la imagen Docker. Las variables de entorno (``ES_HOST``, ``VITE_API_URL``) permiten que cada servicio resuelva las conexiones internas de forma transparente.

4. EVALUACIÓN Y CONCLUSIONES FINALES

4.1. Resultados Obtenidos

A continuación se presenta la tabla consolidada de resultados por módulo, con las métricas principales de cada componente del sistema:

Módulo	Técnica	Métrica principal	Resultado
Anomalías (baseline)	Z-Score Rolling (14d, $2,5\sigma$)	Tasa de anomalía	0,50% · 3.813 alertas · 1.683 productos

Módulo	Técnica	Métrica principal	Resultado
Anomalías (ML)	Isolation Forest (200 árboles)	Tasa de anomalía	0,50% · 3.818 alertas · 59 productos
Anomalías (DL)	Autoencoder LSTM (P99)	Tasa de anomalía	1,00% · 6.929 alertas · 1.300 productos
Complementariedad	Jaccard ZS ↔ IF ↔ AE	Solapamiento	0,006 / 0,000 / 0,005
Predicción cambio	LSTM clasificador	AUC-ROC	0,893 vs 0,771 baseline
Predicción precio	XGBoost regresor	MAPE (cambios reales)	10,75% vs 11,22% naive (+4,2%)
Equivalencias NLP	sentence-transformers	Similitud media	0,844 · 924 pares · brecha +49,0%
Segmentación	K-Means (K=5)	Silhouette Score	0,441 · 76,0% varianza PCA
Shrinkflation	Correlación precio/medida	Casos detectados	20 alertas · severidad media 12,49
Catálogo	Altas/bajas con burn-in	Rotación	615 nuevos · 717 descatalogados
Pipeline ETL	Parquet particionado	Volumen procesado	763.435 obs. · 5.101 productos · 176 días
Despliegue	Docker Compose (4 servicios)	Disponibilidad	ES + Kibana + API + Frontend operativos

4.1.1. Convergencia Intermodular

El resultado más significativo del proyecto no reside en las métricas individuales de cada modelo, sino en la convergencia entre módulos independientes:

- La probabilidad de cambio del LSTM aparece como feature relevante (posición 9/21 en SHAP) del XGBoost, pese a haber sido entrenados de forma completamente independiente.

- La degradación temporal del XGBoost en walk-forward (R^2 de 0,9984 $\rightarrow$ 0,8042) coincide exactamente con el pico de anomalías del Autoencoder en abril-mayo de 2026.
- Los tres detectores de anomalías capturan dimensiones ortogonales del comportamiento anómalo (Jaccard $\approx$ 0), confirmando su complementariedad.

4.2. Análisis Crítico

4.2.1. Fortalezas

1. **Arquitectura modular y extensible:** cada componente opera de forma independiente y genera artefactos reutilizables (archivos Parquet, modelos serializados), lo que permite añadir nuevos módulos sin modificar los existentes.
2. **Honestidad metodológica:** se ha comparado cada modelo contra su baseline correspondiente. Se ha documentado con transparencia que el predictor naive supera al XGBoost en métricas globales (R^2 0,9984 vs 0,7993), explicando que esto es una propiedad del dominio (95,9% de precios estables) y no un fallo del modelo.
3. **Pipeline NLP iterativo y autocrítico:** el cálculo de la brecha de precios pasó por 4 iteraciones de refinamiento, documentando y resolviendo cada sesgo detectado (precios absolutos, unidades incompatibles, monopolio de match) en lugar de presentar solo el resultado final.
4. **Convergencia intermodular no planificada:** la validación cruzada entre el LSTM, el XGBoost y el Autoencoder constituye una forma de triangulación que refuerza la credibilidad de los hallazgos sin haber sido diseñada explícitamente.
5. **Despliegue reproducible:** Docker Compose con 4 servicios permite levantar el stack completo con un único comando, garantizando la reproducibilidad del entorno.

4.2.2. Limitaciones

1. **Fuente única de datos:** el sistema depende exclusivamente del catálogo de Mercadona obtenido mediante un scraper con un código postal concreto. Los hallazgos no son generalizables a otros supermercados ni a otras zonas geográficas sin verificación.
2. **Persistencia dominante en predicción:** en un dominio donde el 95,9% de los precios no cambian en 7 días, cualquier modelo predictivo compite contra un baseline prácticamente óptimo. El valor real del XGBoost se limita al 4,1% de casos con cambio.
3. **Shrinkflation en producto fresco:** el 100% de los casos detectados se concentra en fruta, verdura y pescado, donde la variabilidad de peso por lote puede generar alertas no vinculadas a estrategia comercial deliberada.
4. **Ausencia de búsqueda de hiperparámetros:** no se implementó optimización automatizada (Optuna, Random Search) para ningún modelo. Los hiperparámetros

se seleccionaron por criterio experto y literatura, lo que podría dejar margen de mejora sin explorar.

5. **Escalabilidad limitada del frontend:** la API carga el histórico completo en memoria al arrancar. Para un catálogo de ~4.300 productos esto es viable, pero no escalaría a catálogos de decenas de miles de productos sin migrar a una arquitectura con caché o base de datos analítica.

4.3. Conclusiones

Se concluye que los objetivos planteados en la sección 1.2 se han cumplido en su totalidad:

1. **Pipeline ETL incremental y escalable** (objetivo 1): se construyó un pipeline event-driven con escritura $O(1)$, particionado Parquet y tabla de estado, capaz de procesar 763.435 observaciones sin cuellos de botella de memoria.
2. **Detección de anomalías multi-enfoque** (objetivo 2): los tres detectores (Z-Score, Isolation Forest, Autoencoder) operan de forma complementaria (Jaccard ≈ 0), cubriendo dimensiones temporales, estructurales y secuenciales del comportamiento anómalo.
3. **Sistema de predicción integrado** (objetivo 3): el LSTM clasificador (AUC-ROC 0,893) y el XGBoost regresor (MAPE 10,75% en cambios reales) forman un ensemble complementario donde la señal del LSTM alimenta al XGBoost como feature validada por SHAP.
4. **Pipeline NLP de equivalencias** (objetivo 4): se generaron 924 equivalencias semánticas con similitud media de 0,844, cuantificando una brecha mediana del +49,0% entre marca comercial y marca propia.
5. **IPC personalizado** (objetivo 5): se implementó con 6 perfiles predefinidos, ponderación automática por gasto real y predicción de coste futuro que combina las señales del LSTM y la tendencia histórica.
6. **Análisis complementarios** (objetivo 6): segmentación en 5 perfiles de negocio accionables, 20 alertas de shrinkflation y monitorización de 615 altas y 717 bajas en el catálogo.
7. **API REST y frontend** (objetivo 7): 11 endpoints documentados y 8 vistas interactivas con visualizaciones ECharts que permiten la exploración del sistema sin conocimientos técnicos.
8. **Despliegue Docker** (objetivo 8): stack de 4 servicios con multi-stage build, healthchecks y volúmenes persistentes, reproducible con un único comando.

El proyecto demuestra que es posible construir un sistema de inteligencia de precios integral — desde la ingesta automatizada hasta la visualización interactiva — utilizando herramientas de código abierto y recursos computacionales limitados (GPU de Colab para entrenamiento, CPU local para inferencia). La convergencia intermodular, no planificada sino emergente, constituye la contribución técnica más original del trabajo.

4.4. Líneas de Mejora y Trabajo Futuro

4.4.1. Corto Plazo

- **Optimización de hiperparámetros:** implementar búsqueda automatizada con Optuna o Bayesian Search para los modelos LSTM, XGBoost e Isolation Forest, con validación temporal walk-forward.
- **Reentrenamiento periódico:** automatizar el reentrenamiento de los modelos cuando se detecte degradación (monitorizando el R^2 del XGBoost en ventanas deslizantes).
- **Filtro NLP post-semántico:** añadir un clasificador supervisado ligero que, a partir de pares etiquetados manualmente, filtre los matches semánticamente correctos pero funcionalmente incorrectos (ej. lavavajillas ↔ limpiamáquinas).

4.4.2. Medio Plazo

- **Ampliación a otros supermercados:** integrar datos de Carrefour, Lidl o Alcampo para habilitar comparativas de precios inter-cadena, generalizando los hallazgos más allá de Mercadona.
- **Migración a base de datos analítica:** sustituir la carga en memoria de la API por consultas a ClickHouse o DuckDB para soportar catálogos de mayor volumen.
- **Modelo de series temporales avanzado:** evaluar arquitecturas Transformer (Temporal Fusion Transformer) para la predicción de precios, que podrían capturar mejor las dependencias a largo plazo que el LSTM.
- **Sistema de alertas proactivo:** implementar notificaciones por email o Telegram cuando se detecten anomalías, cambios de precio predichos o casos de shrinkflation.

4.4.3. Largo Plazo

- **Despliegue en la nube:** migrar el stack a Kubernetes o un servicio cloud (AWS ECS, GCP Cloud Run) con escalado automático y CI/CD.
- **Aplicación móvil:** desarrollar una aplicación para el consumidor final que permita escanear productos en tienda y consultar en tiempo real su historial de precios, predicciones y alternativas más baratas.
- **Análisis de elasticidad de demanda:** si se dispusiera de datos de ventas, cruzar los cambios de precio con variaciones de volumen para estimar la elasticidad precio-demanda por categoría.

5. REFERENCIAS

Bibliografía académica y artículos

- [1] Liu, F. T., Ting, K. M. y Zhou, Z.-H. (2008). "Isolation Forest". En: *Proceedings of the Eighth IEEE International Conference on Data Mining (ICDM'08)*, pp. 413–422. DOI: 10.1109/ICDM.2008.17.
- [2] Hochreiter, S. y Schmidhuber, J. (1997). "Long Short-Term Memory". En: *Neural Computation*, vol. 9, n.º 8, pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [3] Chen, T. y Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System". En: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. DOI: 10.1145/2939672.2939785.
- [4] Reimers, N. y Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". En: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. DOI: 10.18653/v1/D19-1410.
- [5] Lundberg, S. M. y Lee, S.-I. (2017). "A Unified Approach to Interpreting Model Predictions". En: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30.
- [6] Arthur, D. y Vassilvitskii, S. (2007). "k-means++: The Advantages of Careful Seeding". En: *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA'07)*, pp. 1027–1035.
- [7] Malhotra, P., Vig, L., Shroff, G. y Agarwal, P. (2015). "Long Short Term Memory Networks for Anomaly Detection in Time Series". En: *Proceedings of the 23rd European Symposium on Artificial Neural Networks (ESANN)*.

Documentación técnica oficial

- [8] The pandas development team. *pandas documentation*. Disponible en: <https://pandas.pydata.org/docs/> [Consulta: mayo 2026].
- [9] Apache Arrow. *PyArrow documentation*. Disponible en: <https://arrow.apache.org/docs/python/> [Consulta: mayo 2026].
- [10] Elastic N.V. *Elasticsearch Reference [7.17]*. Disponible en: <https://www.elastic.co/guide/en/elasticsearch/reference/7.17/index.html> [Consulta: mayo 2026].
- [11] Elastic N.V. *Kibana Guide [7.17]*. Disponible en: <https://www.elastic.co/guide/en/kibana/7.17/index.html> [Consulta: mayo 2026].
- [12] TensorFlow. *TensorFlow API documentation (v2.x)*. Disponible en: https://www.tensorflow.org/api_docs [Consulta: mayo 2026].
- [13] XGBoost developers. *XGBoost documentation*. Disponible en: <https://xgboost.readthedocs.io/> [Consulta: mayo 2026].
- [14] scikit-learn developers. *scikit-learn user guide*. Disponible en: https://scikit-learn.org/stable/user_guide.html [Consulta: mayo 2026].
- [15] UKPLab. *Sentence-Transformers documentation*. Disponible en: <https://www.sbert.net/> [Consulta: mayo 2026].

Frameworks y herramientas

[16] Pallets Projects. *Flask documentation*. Disponible en: <https://flask.palletsprojects.com/> [Consulta: mayo 2026].

[17] Vue.js. *Vue 3 documentation*. Disponible en: <https://vuejs.org/guide/> [Consulta: mayo 2026].

[18] Vite. *Vite documentation*. Disponible en: <https://vitejs.dev/guide/> [Consulta: mayo 2026].

[19] Apache ECharts. *ECharts documentation*. Disponible en: <https://echarts.apache.org/en/index.html> [Consulta: mayo 2026].

[20] Docker Inc. *Docker Compose documentation*. Disponible en: <https://docs.docker.com/compose/> [Consulta: mayo 2026].

[21] Nginx Inc. *Nginx documentation*. Disponible en: <https://nginx.org/en/docs/> [Consulta: mayo 2026].

[22] GitHub. *GitHub Actions documentation*. Disponible en: <https://docs.github.com/en/actions> [Consulta: mayo 2026].

[23] Gorakhti, B. *watchdog — Python API library and shell utilities to monitor file system events*. Disponible en: <https://python-watchdog.readthedocs.io/> [Consulta: mayo 2026].

Modelos preentrenados

[24] Reimers, N. *paraphrase-multilingual-MiniLM-L12-v2*. Hugging Face Model Hub. Disponible en: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2> [Consulta: mayo 2026].

6. GLOSARIO DE TÉRMINOS Y ACRÓNIMOS

Término / Acrónimo	Definición
AE	Autoencoder. Red neuronal que aprende a comprimir y reconstruir datos, utilizada para detección de anomalías por error de reconstrucción.
API	<i>Application Programming Interface</i> . Interfaz de programación que permite la comunicación entre componentes de software.
AUC-ROC	<i>Area Under the Receiver Operating Characteristic Curve</i> . Métrica de evaluación de clasificadores que mide la capacidad de discriminación entre clases.
Axios	Cliente HTTP basado en promesas para JavaScript, utilizado en el frontend para la comunicación con la API REST.

Término / Acrónimo	Definición
Batch Normalization	Técnica de regularización en redes neuronales que normaliza las activaciones de cada capa durante el entrenamiento.
Burn-in	Periodo de quemado. Ventana temporal inicial de un scraper donde el catálogo base aún no se ha estabilizado.
CI/CD	<i>Continuous Integration / Continuous Deployment</i> . Práctica de automatización de pruebas y despliegue de software.
Composition API	Paradigma de desarrollo de Vue 3 que organiza la lógica de componentes mediante funciones composables, en lugar del sistema de opciones de Vue 2.
CORS	<i>Cross-Origin Resource Sharing</i> . Mecanismo de seguridad web que permite peticiones entre dominios distintos.
Coseno (Similitud)	Métrica que mide el ángulo entre dos vectores en un espacio de alta dimensionalidad. Valores cercanos a 1 indican alta similitud semántica.
CSV	<i>Comma-Separated Values</i> . Formato de texto plano para almacenar datos tabulares.
Data Lake	Repositorio centralizado de datos en bruto o procesados, almacenados en su formato nativo.
DL	<i>Deep Learning</i> . Subconjunto del aprendizaje automático basado en redes neuronales profundas.
Docker	Plataforma de contenerización que empaqueta aplicaciones con sus dependencias para garantizar la reproducibilidad.
Dropout	Técnica de regularización que desactiva aleatoriamente neuronas durante el entrenamiento para prevenir el sobreajuste.
Early Stopping	Estrategia de parada temprana del entrenamiento cuando la métrica de validación deja de mejorar.
ECharts	Librería JavaScript de visualización de datos interactivos de código abierto.

Término / Acrónimo	Definición
Elasticsearch (ES)	Motor de búsqueda y analítica distribuido basado en Apache Lucene.
Embedding	Representación vectorial densa de un dato (texto, imagen) en un espacio continuo de dimensionalidad fija, donde la proximidad geométrica refleja similitud semántica.
ETL	<i>Extract, Transform, Load</i> . Proceso de extracción, transformación y carga de datos.
F1-Score	Media armónica de Precision y Recall. Métrica de evaluación para clasificadores con clases desbalanceadas.
Feature Engineering	Proceso de creación de variables derivadas a partir de datos brutos para mejorar el rendimiento de los modelos.
GPU	<i>Graphics Processing Unit</i> . Procesador especializado utilizado para acelerar el entrenamiento de modelos de DL.
Gradient Boosting	Técnica de ensemble que construye modelos de forma secuencial, donde cada nuevo modelo corrige los errores residuales del anterior. XGBoost es una implementación optimizada.
IF	<i>Isolation Forest</i> . Algoritmo de detección de anomalías basado en la facilidad de aislar observaciones atípicas.
IPC	Índice de Precios al Consumo. Indicador económico que mide la evolución de los precios de una cesta de bienes.
Jaccard	Índice de similitud entre conjuntos, definido como la intersección dividida por la unión.
K-Means	Algoritmo de clustering que agrupa observaciones en K clusters minimizando la inercia intra-cluster.
Kibana	Interfaz de visualización y exploración de datos indexados en Elasticsearch.

Término / Acrónimo	Definición
Lazy Loading	Técnica de carga diferida que retrasa la carga de módulos o recursos hasta que son necesarios, reduciendo el tiempo de carga inicial de la aplicación.
LSTM	<i>Long Short-Term Memory</i> . Tipo de red neuronal recurrente diseñada para aprender dependencias temporales a largo plazo.
MAE	<i>Mean Absolute Error</i> . Error absoluto medio entre predicciones y valores reales.
MAPE	<i>Mean Absolute Percentage Error</i> . Error porcentual absoluto medio.
ML	<i>Machine Learning</i> . Aprendizaje automático.
MSE	<i>Mean Squared Error</i> . Error cuadrático medio, función de pérdida habitual en regresión y autoencoders.
Multi-stage build	Técnica de Docker que compila una aplicación en una etapa y copia solo los artefactos al contenedor final, reduciendo el tamaño de la imagen.
NDJSON	<i>Newline Delimited JSON</i> . Formato de exportación de objetos Kibana (dashboards, visualizaciones).
Nginx	Servidor web de alto rendimiento utilizado como servidor de archivos estáticos y proxy inverso, redirigiendo peticiones a servicios backend internos.
NLP	<i>Natural Language Processing</i> . Procesamiento del Lenguaje Natural.
Outlier	Observación atípica que se desvía significativamente del patrón general de los datos. Puede ser legítimo (producto premium) o erróneo (error de catalogación).
Parquet	Formato de almacenamiento columnar de Apache, optimizado para lectura analítica y compresión.

Término / Acrónimo	Definición
PCA	<i>Principal Component Analysis</i> . Técnica de reducción de dimensionalidad que proyecta datos en componentes de máxima varianza.
Precision / Recall	Precision: proporción de predicciones positivas que son correctas. Recall: proporción de positivos reales que el modelo detecta. Métricas complementarias en clasificación.
Predicate Pushdown	Optimización de lectura que filtra datos a nivel de motor de almacenamiento antes de cargarlos en memoria.
R²	Coefficiente de determinación. Mide la proporción de varianza explicada por un modelo de regresión.
REST	<i>Representational State Transfer</i> . Estilo arquitectónico para servicios web que utiliza métodos HTTP estándar (GET, POST, PUT, DELETE) sobre recursos identificados por URLs.
RMSE	<i>Root Mean Squared Error</i> . Raíz del error cuadrático medio.
SCRUM	Marco de trabajo ágil para la gestión de proyectos basado en sprints e incrementos funcionales.
SHAP	<i>SHapley Additive exPlanations</i> . Método de interpretabilidad de modelos basado en la teoría de juegos cooperativos.
Shrinkflation	Reduflación. Práctica comercial de reducir el tamaño de un producto manteniendo su precio.
Silhouette Score	Métrica de validación de clustering que mide la cohesión intra-cluster y la separación inter-cluster.
SKU	<i>Stock Keeping Unit</i> . Código único que identifica un producto específico en un formato y envase concretos dentro de un catálogo comercial.
Snappy	Algoritmo de compresión rápida desarrollado por Google, utilizado como codec por defecto en ficheros Parquet. Prioriza velocidad sobre ratio de compresión.

Término / Acrónimo	Definición
SPA	<i>Single Page Application</i> . Aplicación web que carga una única página HTML y actualiza dinámicamente su contenido.
StandardScaler	Transformador de scikit-learn que normaliza las features restando la media y dividiendo por la desviación típica (estandarización z-score).
t-SNE	<i>t-Distributed Stochastic Neighbor Embedding</i> . Técnica de reducción de dimensionalidad para visualización de datos de alta dimensionalidad.
Vite	Herramienta de build para aplicaciones web modernas con soporte de <i>Hot Module Replacement</i> (HMR) para desarrollo rápido y compilación optimizada para producción.
vue-router	Librería oficial de enrutamiento de Vue.js que gestiona la navegación entre vistas en una SPA, con soporte de <i>lazy loading</i> de componentes.
Walk-forward	Método de validación temporal expansiva donde el conjunto de entrenamiento crece acumulativamente en cada fold.
Watchdog	Librería Python para monitorizar eventos del sistema de ficheros en tiempo real.
XGBoost	<i>eXtreme Gradient Boosting</i> . Implementación eficiente de gradient boosting basada en árboles de decisión.
Z-Score	Número de desviaciones típicas que una observación dista de la media. Usado como detector de anomalías temporales.